

Outline

- ▶ Introduction and applications
- ▶ Traditional methods
 - ▶ Text Representation
 - ▶ Text Preprocessing Steps
- ▶ Advanced techniques for text representation (neural embeddings)
 - ▶ Neural network-based embeddings
 - ▶ Static Embeddings
 - ▶ Contextual Embeddings with Transformers
 - ▶ Text classification with embeddings

Motivation for Contextual Word Embedding

- ▶ Word2Vec and its variants are static word embedding models
 - ▶ The embedding of a word is fixed after training and does not change with its context.
- ▶ However, a word can have different meaning, e.g.,
 - ▶ Bank of Montreal vs. Bank of St. Lawrence River.
 - ▶ The chicken did not cross the road because it was too tired.
vs.
The chicken did not cross the road because it was too wide,

Contextual Word Embeddings

- ▶ A representation of meaning of a word should be different in different contexts!
- ▶ Contextual Embedding:
 - ▶ A word has different vectors that expresses different meanings depending on the surrounding words
 - ▶ Consider other words in the sentence or document when computing the embedding of a word
- ▶ How to compute contextual embeddings?
 - ▶ Self-Attention.

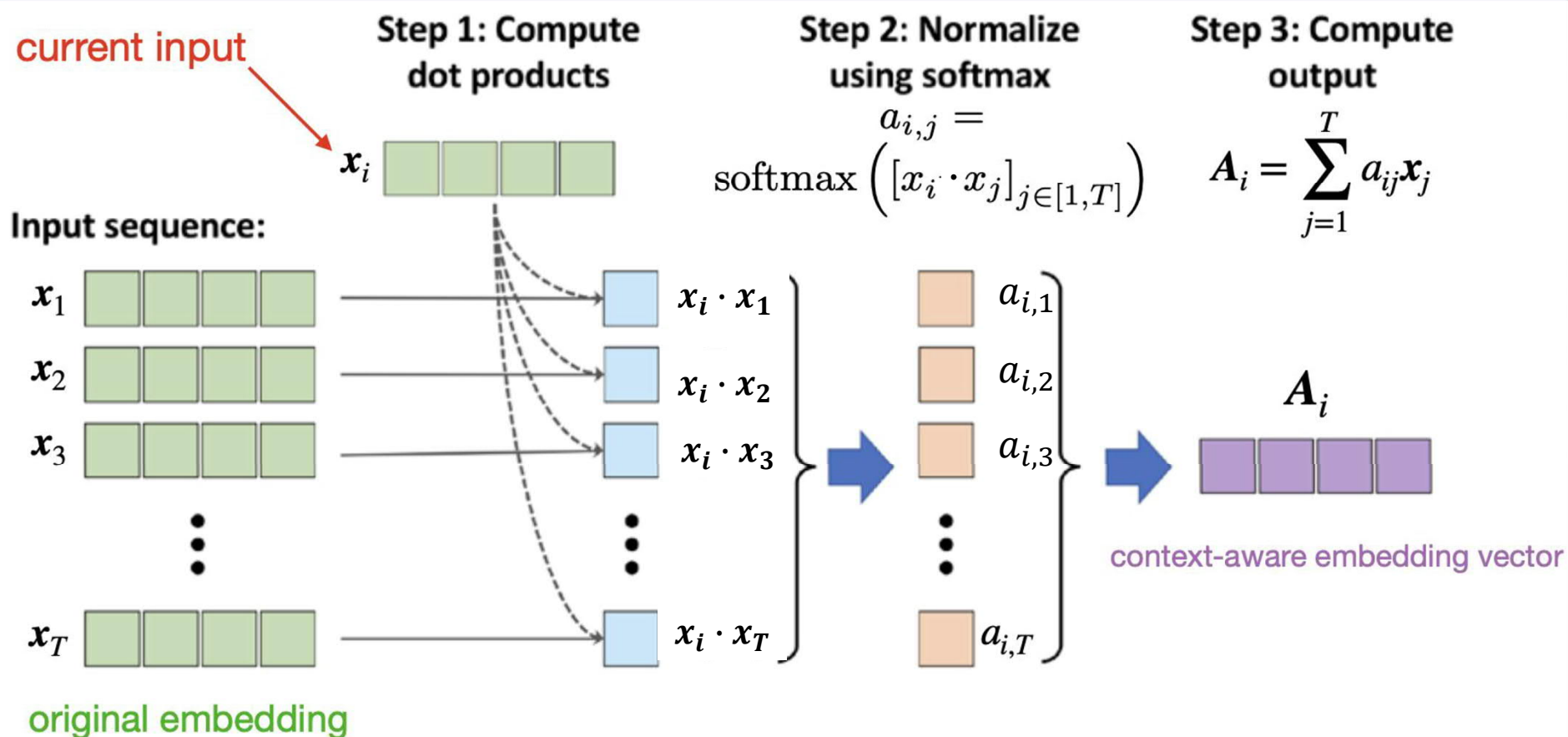
Self-Attention

- ▶ Attention is a mechanism in neural networks that
 - ▶ allows the model to selectively focus on specific parts of the input data when producing an output,
 - ▶ Assigns dynamic importance weights to different elements.
 - ▶ Mimics human cognitive attention
 - ▶ Crucial for handling long sequences and capturing long-range dependencies in data, such as the relationships between words in a sentence.
- ▶ Self-attention is a mechanism that
 - ▶ Computes a contextualized representation for each word in the input sequence by allowing it to "look at" (i.e., attend to) and weigh the importance of all other words in that same sequence.
 - ▶ The embedding of a word is a weighted sum of the embeddings of all other words in the sequence.

Self-Attention – Basic, No-Trainable Weights

- Given an input sentence $(x_1, x_2, \dots, x_T)$, where x_i is a word/token (represented by their embeddings), convert them into context-aware embeddings $(A_1, A_2, \dots, A_T)$

Self-Attention (Basic-No weights)



Self-Attention with Trainable Weight Matrices

- ▶ Train three weight matrices (W^q , W^k , W^v) to convert each word (or token) embedding x_i into three vectors q_i , k_i , v_i , representing 3 separate roles of x_i

- ▶ **Query vector:** $q_i = x_i W^q$

- ▶ Represents the token's "question" presented to other tokens.

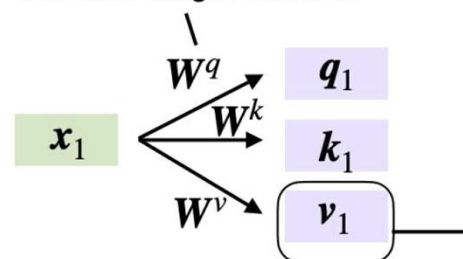
- ▶ **Key vector:** $k_i = x_i W^k$

- ▶ Represents the token's "answer" to other tokens' questions.

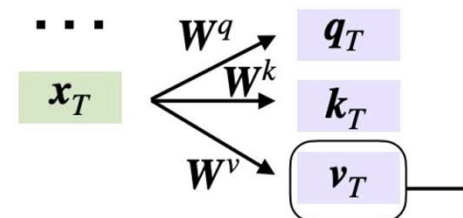
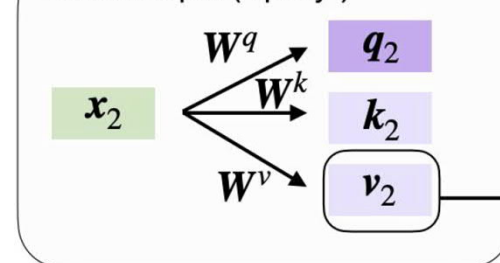
- ▶ **Value vector:** $v_i = x_i W^v$

- ▶ Represents the actual information the token carries.

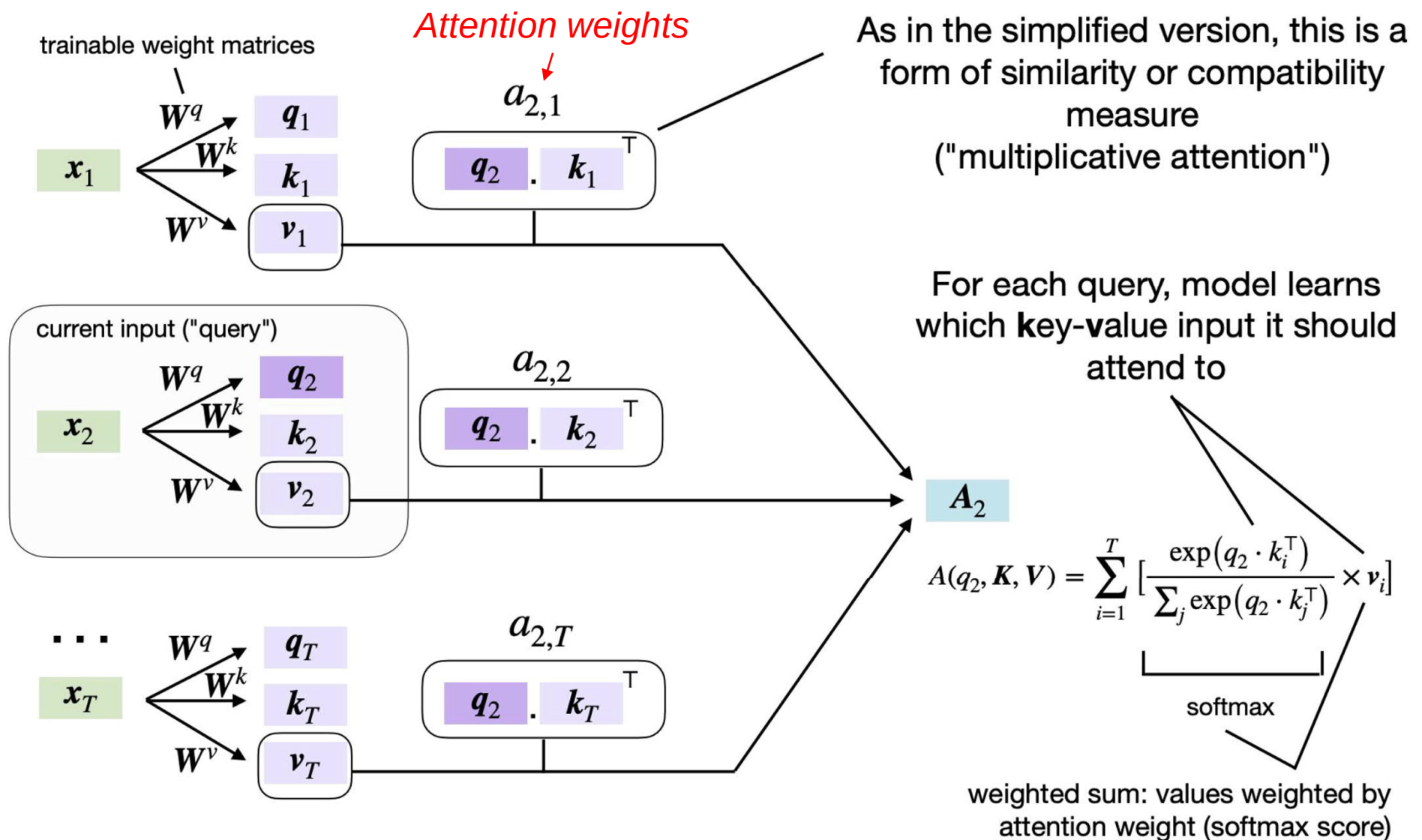
trainable weight matrices



current input ("query")



Self-Attention with Trainable Weights

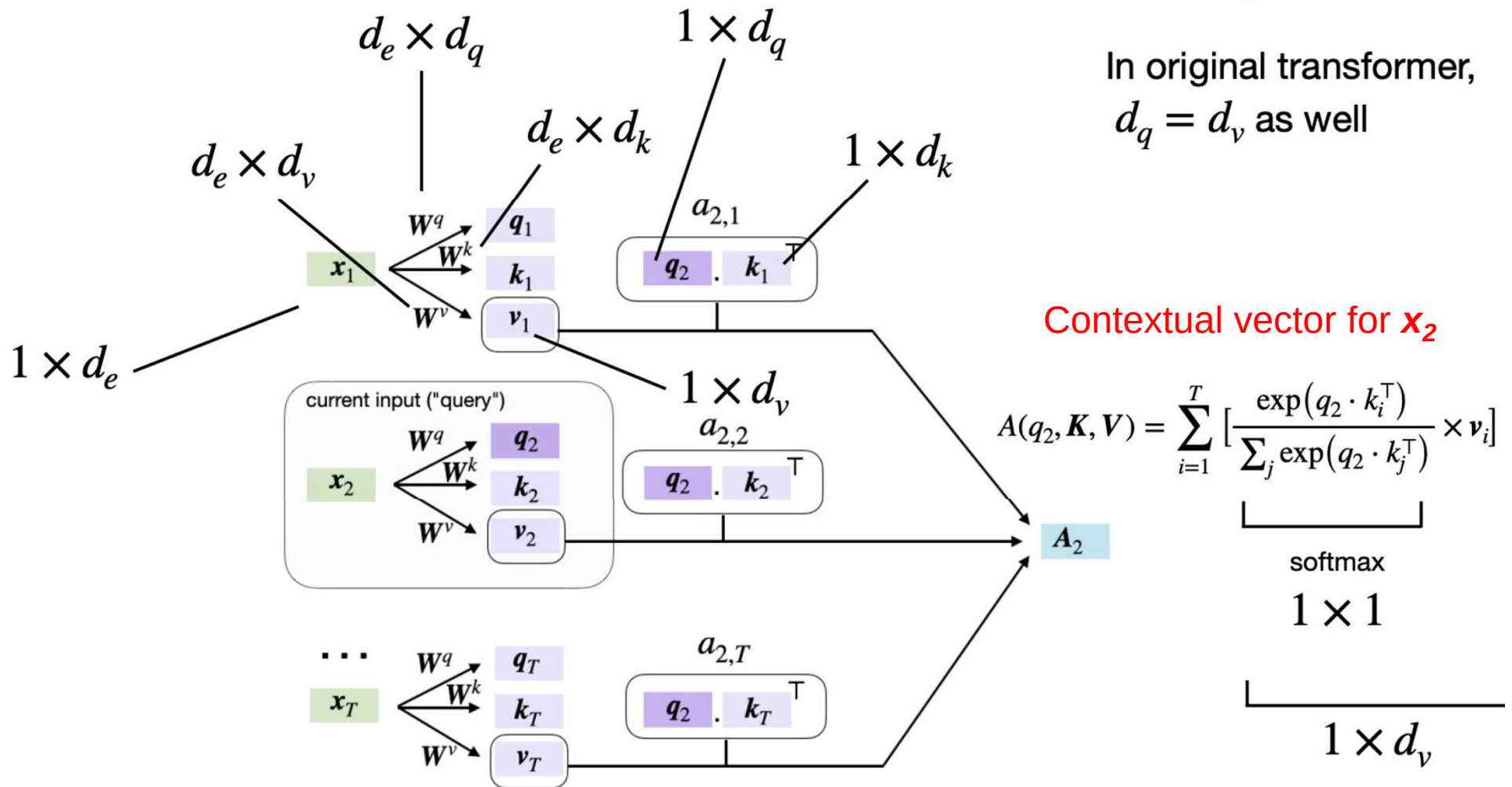


Dimensions of Matrices and Vectors

d_e = embedding size (original transformer = 512)

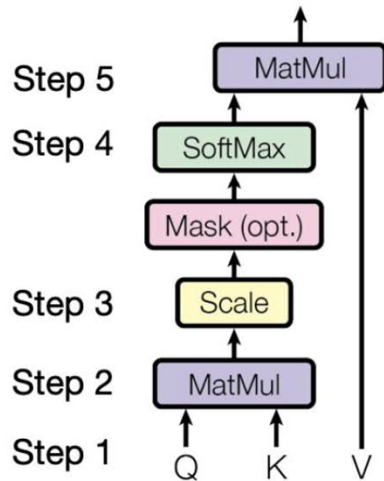
where $d_q = d_k$

In original transformer,
 $d_q = d_v$ as well

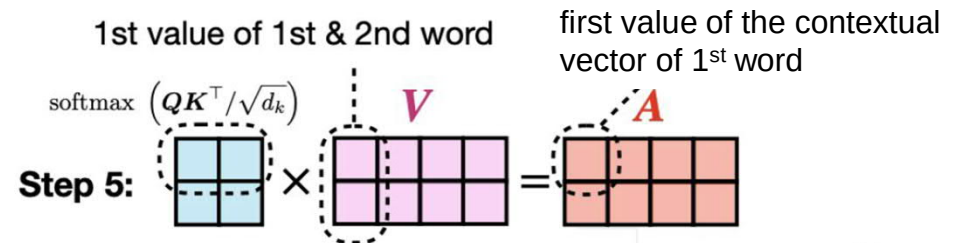
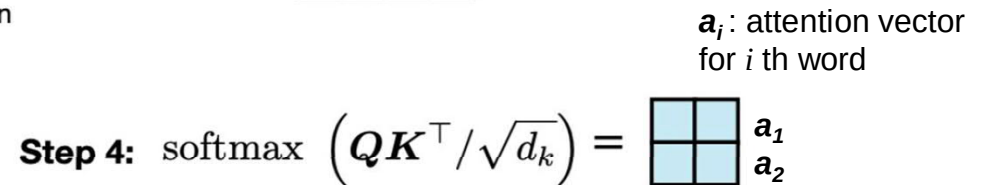
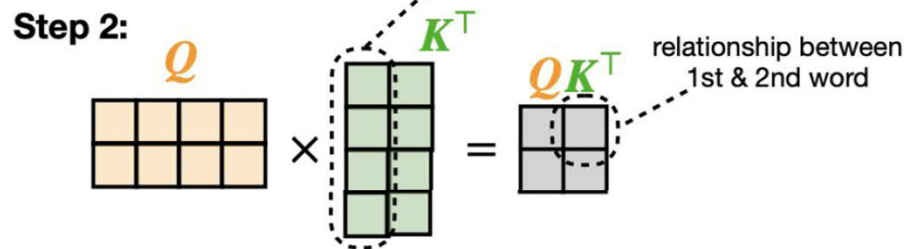
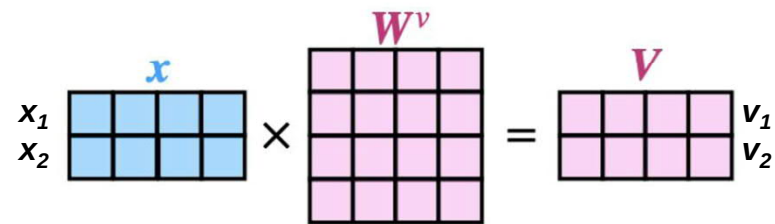
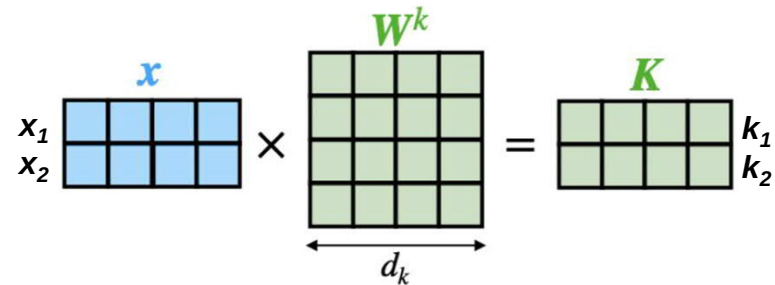
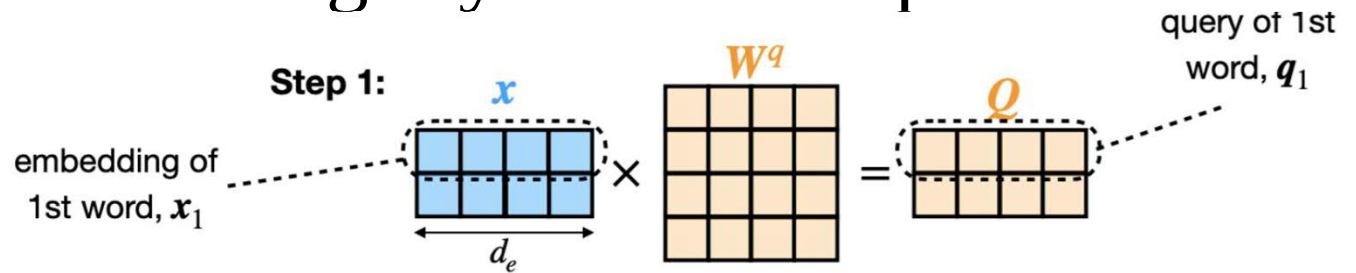


Actual Attention: Slightly More Complicated

Scaled Dot-Product Attention

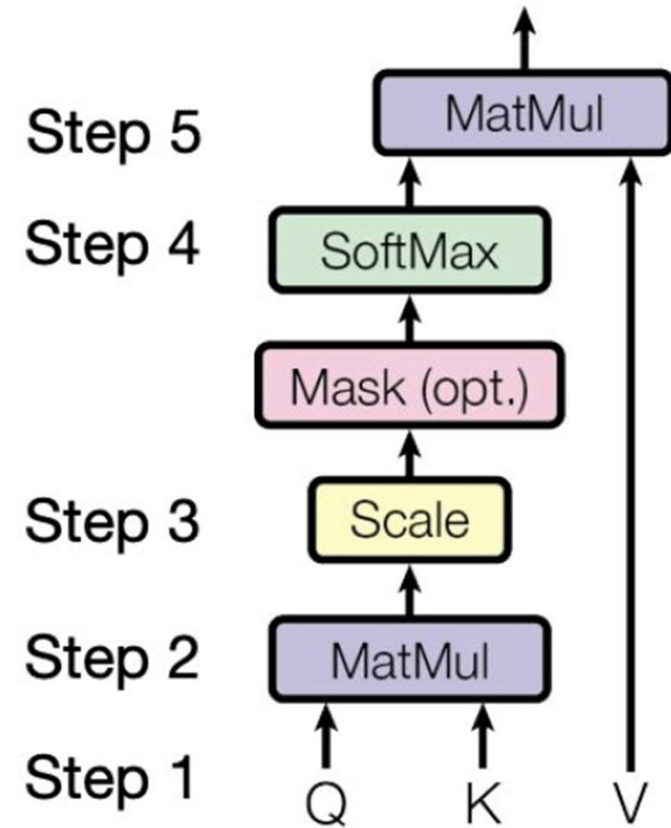


Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L. and Polosukhin, I., 2017. Attention Is All You Need.



Attention Formula (Single Head)

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



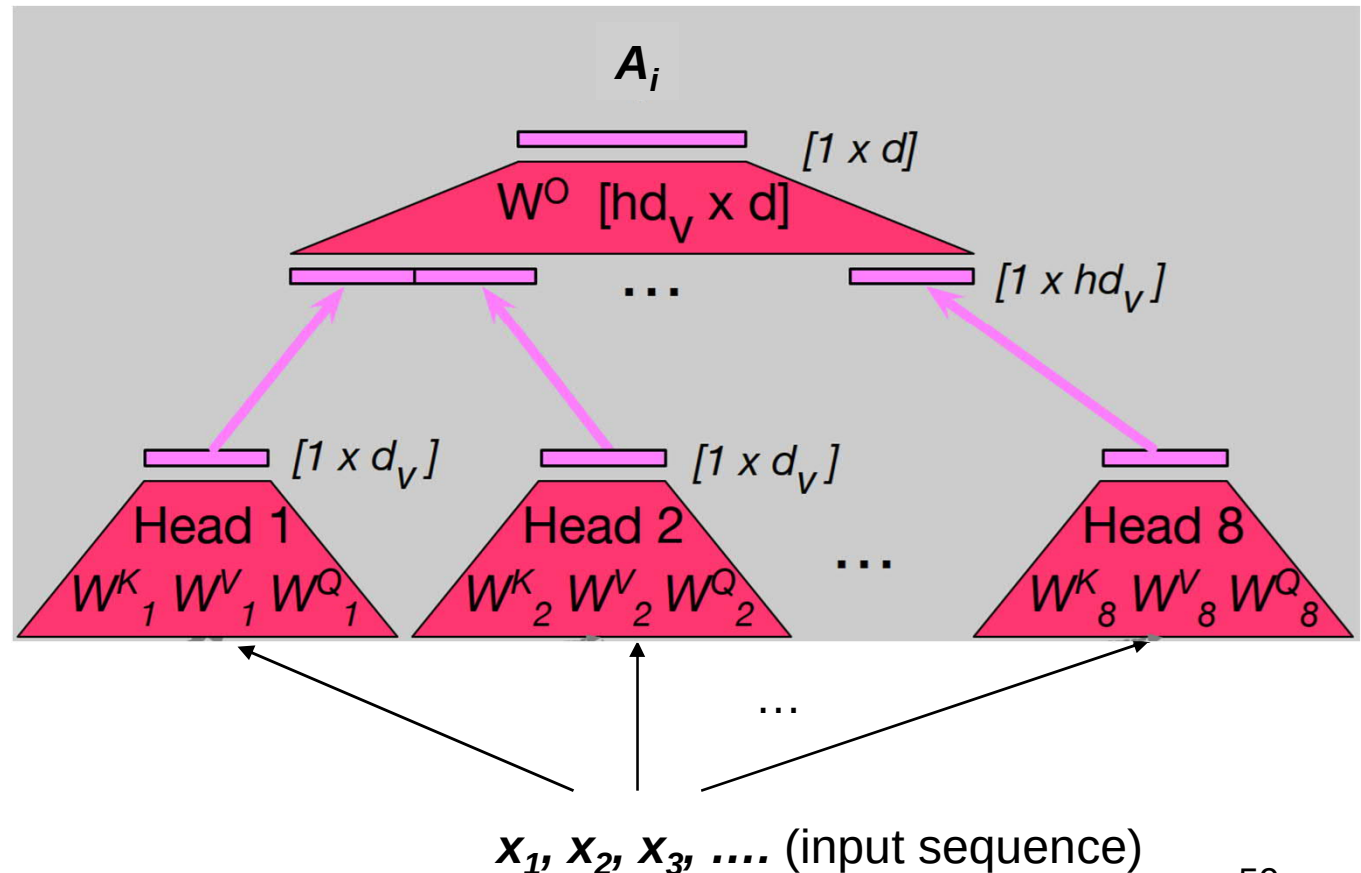
Multi-Head Attention

- ▶ Project the embeddings of input words multiple times, each with a set of W^K , W^V and W^Q .
- ▶ Results are concatenated and project to a smaller dimension

Project down to d dimension

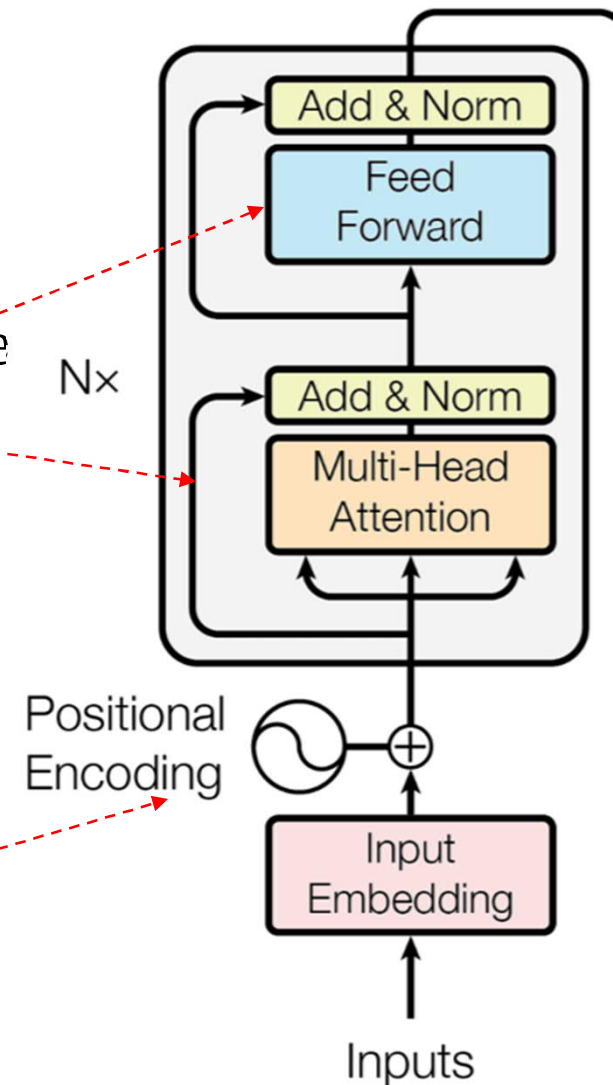
Concatenate outputs

Each head attend differently to context (considering different aspects)



Transformer Encoder

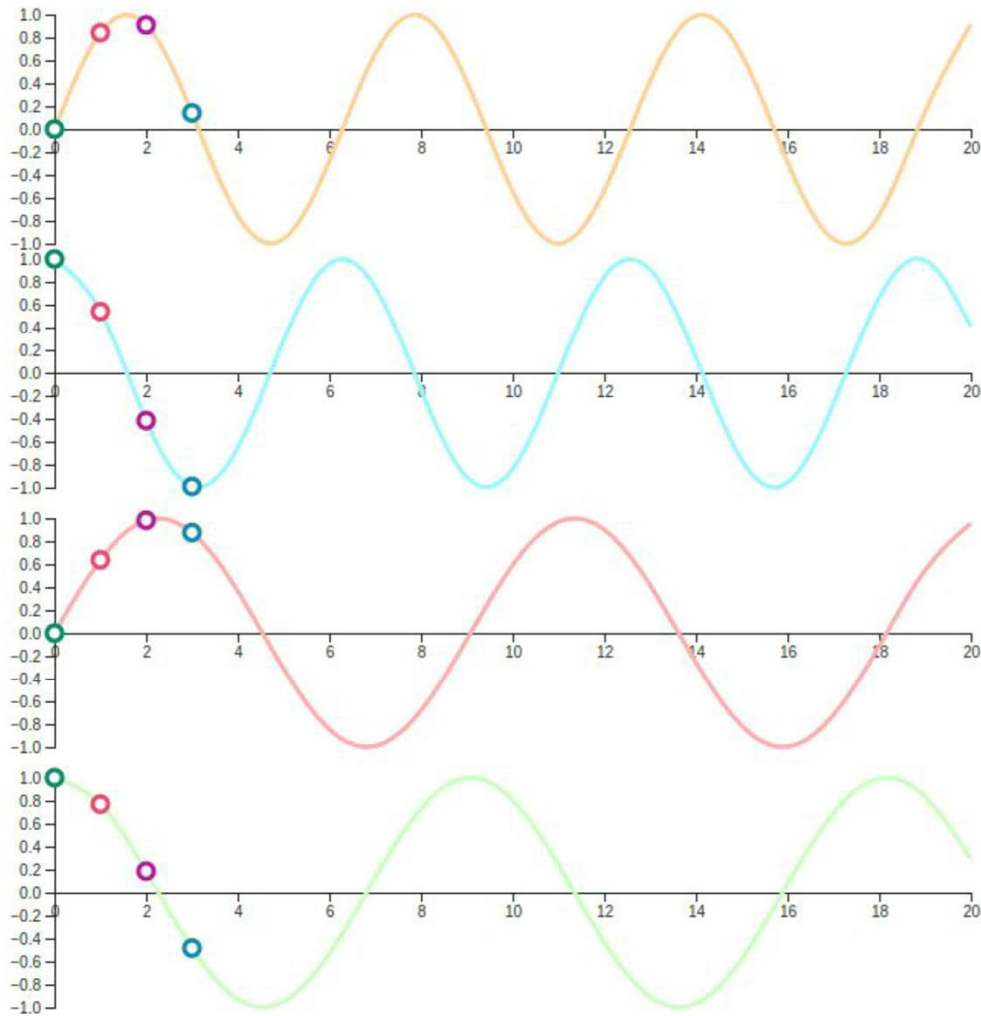
- ▶ Multi-head attention is just the main part of the Transform Encoder block
- ▶ A residual connection is added to mitigate gradient vanishing
- ▶ A feed-forward layer is added after a normalization layer
 - ▶ Converts the contextual embedding of each word using a non-linear activation function
- ▶ At the input part, positional encoding is added to the input embedding



Positional Encoding

- ▶ Provides information about the order of tokens in a sequence, ensuring the Transformer understands word order despite processing words/tokens in parallel.
- ▶ Use sine and cosine functions to create unique positional embeddings that are **added** to the input token embeddings
- ▶ Without positional encoding, the model would treat the input as a bag of unordered words/tokens

Positional Encoding



p0	p1	p2	p3	
0.000	0.841	0.909	0.141	i=0
1.000	0.540	-0.416	-0.990	i=1
0.000	0.638	0.983	0.875	i=2
1.000	0.770	0.186	-0.484	i=3

Positional Encoding

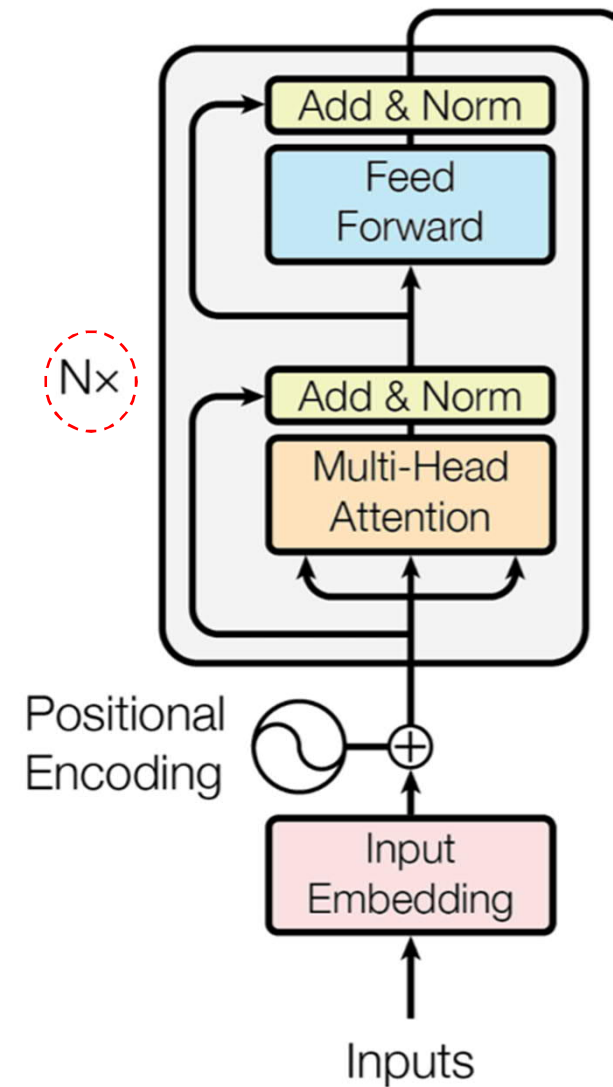
$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

- pos = token position (0, 1, 2, ...)
- i = dimension index (0, 1, 2, ...)
- d_{model} = embedding dimension (e.g., 512)

Transformer Encoder (N Blocks)

- ▶ Can have N blocks
- ▶ BERT-Small: 12 blocks
- ▶ BERT-Large: 24 blocks



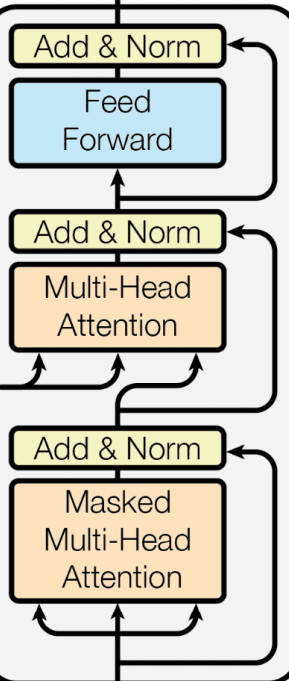
Transformer Architecture

Decoder

Output
Probabilities

Softmax

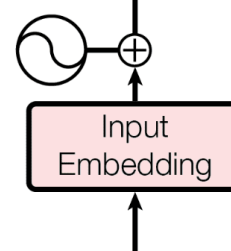
Linear



Encoder

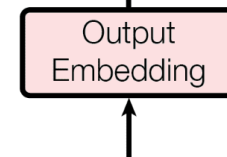
$N \times$

Positional
Encoding



Inputs

Positional
Encoding



Outputs
(shifted right)

- ▶ **Encoder**
 - ▶ Processes the input sequence
 - ▶ Creates a contextualized embedding for each input token
 - ▶ Captures the meaning of the input sequence and relationships between words.
- ▶ **Decoder**
 - ▶ Generates the output sequence step-by-step, using
 - ▶ Encoder's contextualized representations
 - ▶ Its own previously generated tokens

How to Train W^q , W^k , W^v

- ▶ Depends on the model
- ▶ For the BERT model:
 - ▶ Use a huge set of corpora
 - ▶ Books Corpus: This dataset contains
 - ▶ approximately 800 million words from over 11,000 unpublished books.
 - ▶ a variety of genres, which helps the model learn diverse language patterns.
 - ▶ English Wikipedia: This dataset consists of English Wikipedia articles,
 - ▶ totaling around 2.5 billion words.
 - ▶ a broad range of topics and formal writing styles, contributing to the model's general language understanding.
 - ▶ Trained with unlabeled text on two NLP tasks:
 - ▶ Masked language model
 - ▶ Next sentence prediction

Outline

- ▶ Introduction and applications
- ▶ Traditional methods
 - ▶ Text Representation
 - ▶ Text Preprocessing Steps
- ▶ Advanced techniques for text representation (neural embeddings)
 - ▶ Neural network-based embeddings
 - ▶ Static Embeddings
 - ▶ Contextual Embeddings with Transformers
 - ▶ Text classification with embeddings

Usefulness of Word Embeddings

- ▶ Can be used to compute similarity between words

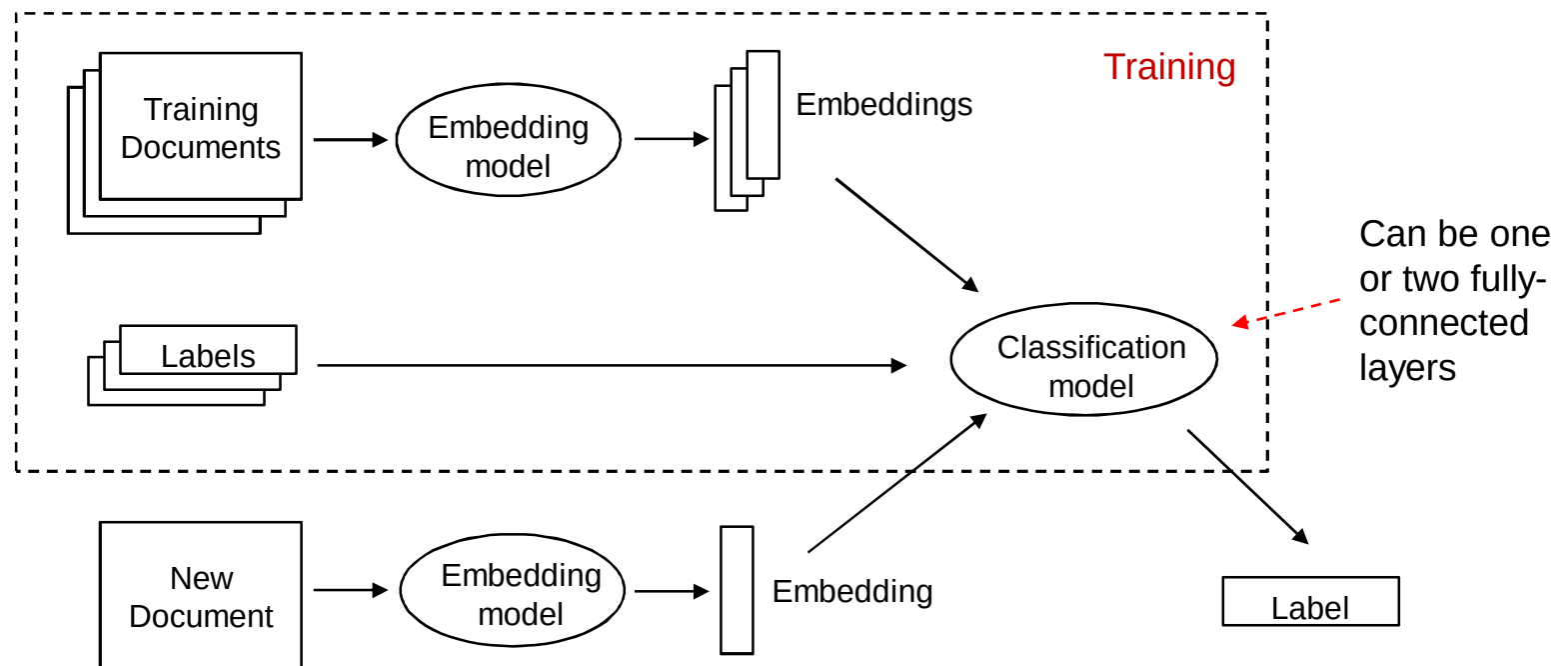
$$\text{cosine_similarity}(w_i, w_j) = \frac{\vec{w}_i \cdot \vec{w}_j}{\|\vec{w}_i\| \|\vec{w}_j\|}$$

where w_i and w_j are words and $\vec{w}_i$ and $\vec{w}_j$ are word vectors

- ▶ Useful for tasks, such as **text matching**, **information retrieval**, etc.
- ▶ Building block for longer text embedding
 - ▶ **Sentence embedding** (Sent2Vec, Sentence-BERT, etc.)
 - ▶ **Paragraph/Document** Embedding (doc2vec)
- ▶ Used in many other NLP tasks, such as
 - ▶ Text classification
 - ▶ Machine translation
 - ▶ Summarization
 - ▶ Question answering

Text Classification with Embeddings

- ▶ Objective is to learn to classify a document
- ▶ Two main steps:
 - ▶ Compute an embedding for the document
 - ▶ Classify the embedding into a class



How to Get Document/Text Embeddings

- ▶ There are a few ways to do so, e.g.,
 - 1) Compute a *document vector* with an average of word/token embeddings (classic)
 - 2) **Fine-tune** a pre-trained model (e.g., BERT, RoBERTa, Longformer) to get a *sentence/text embedding*.
 - 2) **Use a pre-trained model for** *sentence/text embeddings* (e.g., SBERT, SimCSE, etc.)

(1) Classic Methods for Computing Document Vector with Word/Token Embeddings

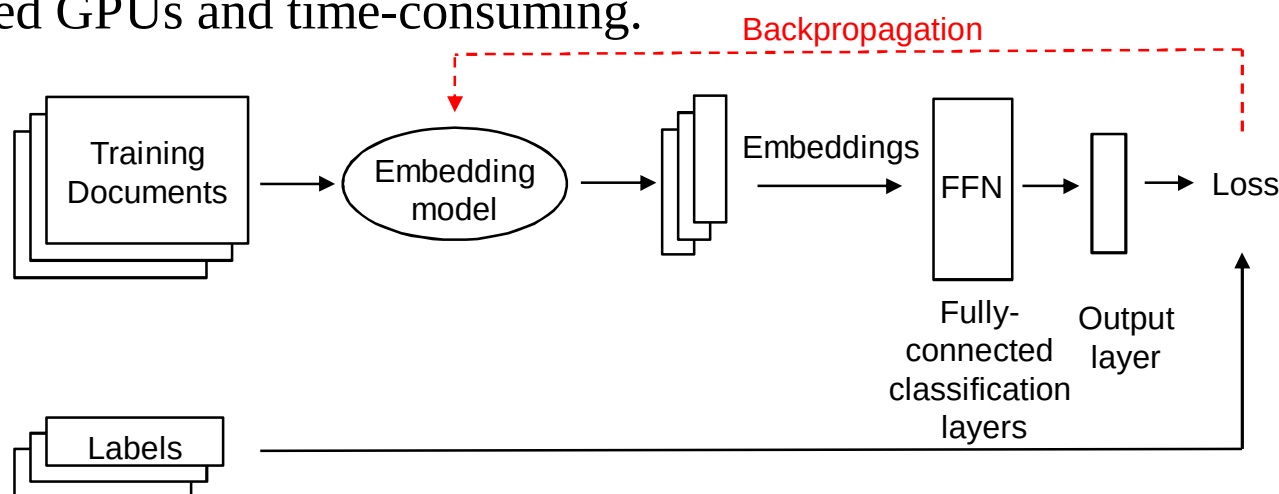
- ▶ Simple averaging on word embeddings
 - ▶ Average the embeddings of the words occurring in the document
- ▶ TF-IDF weighted averaging on word embeddings
 - ▶ Using the TF-IDF value of a word in the document as the weight for the word when averaging word embeddings
- ▶ Word embeddings can be
 - ▶ Pre-trained with a large text corpus (e.g., Word2Vec, GloVe, FastText)
 - ▶ Fine-tuned with the training data for the classification task
 - ▶ Freshly trained with the training data (seldom-used)

(2) Fine-tune Pre-trained Sentence/Text Embedding Model

- ▶ Some pre-trained transformer-based embedding models, like BERT, RoBERTa, Longformer, etc.
 - ▶ Put a special token [CLS] at the beginning of an input sentence/text
 - ▶ Can use the [CLS] token embedding as the sentence/text embedding
- ▶ Their input sizes (m tokens) vary:
 - ▶ BERT, RoBERTa: 512 tokens
 - ▶ Longformer: 4096 tokens
- ▶ If document fits the input size, the [CLS] token embedding is the document embedding
- ▶ If not,
 - ▶ take the first or middle m tokens of the document to classify
 - ▶ or chunk the document into chunks and average the predictions over the chunks.

(2) Fine-tune Pre-trained Sentence/Text Embedding Model (*Cont'd*)

- ▶ These models (BERT, RoBERTa, Longformer) are trained
 - ▶ for producing contextual word/token embeddings with self-supervised tasks masked token prediction and next sentence prediction.
- ▶ For classification, it is better to **fine-tune** the model with the classification training data to obtain good accuracy.
 - ▶ Need GPUs and time-consuming.



(3) Use a Pre-trained Sentence/Text Embedding without Fine-tuning

- ▶ Some transformer models are trained to produce sentence/text embeddings that better capture the meaning
 - ▶ Sentence-BERT (SBERT) – most commonly used
 - ▶ A framework or method that fine-tunes BERT or its variants (e.g., RoBERTa, MPNet, etc.) to produce fixed-size sentence/text embeddings.
 - ▶ Uses two (or three) identical BERT-like models with shared weights to process sentence pairs (or triplets) independently
 - ▶ Trained on multiple loss functions such as contrastive loss, ensuring that positive pairs (similar sentences) are closer in the vector space than negative pairs (dissimilar sentences) by a certain margin.
 - ▶ Other methods: SimCSE, USE

(3) Use a Pre-trained Sentence/Text Embedding without Fine-tuning (*Cont'd*)

- ▶ A popular library for pre-trained sentence/text embedding models are Hugging Face's
 - ▶ *sentence-transformers*
- ▶ The *sentence-transformers* library
 - ▶ Can efficiently produce sentence, text, and word embeddings using pre-trained transformer models
 - ▶ Contain many base models:
 - ▶ https://www.sbert.net/docs/pretrained_models.html

(3) Use a Pre-trained Document/Text Embedding without Fine-tuning (*Cont'd*)

▶ Sample code using sentence_transformers

```
!pip install -U sentence_transformers

from sentence_transformers import SentenceTransformer

# Initialize the SBERT model
model = SentenceTransformer('bert-base-nli-mean-tokens') # Example base model based on BERT-Base

# List of reviews, each review can be a single or multiple sentences
reviews = [
    "Great product, fast shipping!",
    "Not happy with the purchase. The item arrived late and was damaged.",
    "Excellent quality, would buy again."
]

# Generate embeddings for each review
review_embeddings = model.encode(reviews)
```

Each element in `review_embeddings` is the vector representation of the corresponding review

Summary

- ▶ Text classification has many applications
- ▶ Traditional methods
 - ▶ Text Representation
 - ▶ Text Preprocessing Steps
 - ▶ Can use any classification models
- ▶ Advanced techniques for text representation and classification
 - ▶ Neural network-based embeddings
 - ▶ Static Embeddings
 - ▶ Contextual Embeddings with Transformers
 - ▶ Text classification with embeddings